



BrightLearn
AI & DATA SKILLS

2026 EDITION · BEGINNER'S GUIDE

SQL

Master Guide

The language every database speaks. Learn to query, build, and modify data in MySQL — from your very first SELECT statement to joining tables with confidence.

TOPIC

Beginner SQL & MySQL

SECTIONS

10 chapters

LEVEL

Beginner-friendly

INCLUDES

Syntax · Examples · Diagrams



00

What is SQL?

Every database in the world — from your bank's transaction system to your favourite app's user data — uses one language to talk to its data. That language is SQL.

DEFINITION

SQL

SQL — **Structured Query Language** — is the language we use to communicate with databases. We use it to create, read, update, and delete data. Almost every database product on the planet, from MySQL to PostgreSQL to Oracle, understands SQL.

DEFINITION

Database

A **database** is an organised collection of data — typically split into tables that relate to each other. A **DBMS (Database Management System)** is the software that runs the database for you. Common examples: MySQL, Oracle, SQL Server, PostgreSQL, SQLite.

★ WHY MySQL FOR LEARNING SQL?

- **Free** — open source and costs nothing to install.
- **Popular** — used by huge companies, so the skills transfer everywhere.
- **Easy to use** — clean syntax, a friendly Workbench, and forgiving defaults.
- **Great for beginners** — the perfect first DBMS to learn the language on.

★ THE MENTAL MODEL — SQL vs MySQL

SQL is the language. MySQL is one of many software products that speaks it. Think of SQL like English, and MySQL like a particular pen you write English with. Once you can write SQL, you can use it in PostgreSQL, SQL Server, Oracle, or any other DBMS with only small dialect differences.





01 Getting Ready to Code

Before you write your first `SELECT`, you need two things installed: MySQL Server (which stores and runs your data) and MySQL Workbench (the visual tool where you'll write and test your code).

DEFINITION

MySQL Server

The actual database engine. It runs in the background, stores your databases on disk, and waits for SQL queries to arrive. You will rarely look at it directly — you talk to it through Workbench.

DEFINITION

MySQL Workbench

A graphical app that gives you a place to write SQL, run it against your server, and see the results. This is where you will spend most of your time as a beginner.

★ THE FIVE WORKBENCH AREAS TO KNOW

- **Text Editor** — where you type your SQL code.
- **Execute Selected** — runs only the SQL you have highlighted.
- **Execute Statement** — runs the single statement under your cursor.
- **Output Window** — shows messages, errors, and execution status.
- **Result Grid** — displays the rows your query returned, like a spreadsheet.

YOUR FIRST MySQL CODE

```
-- Using SELECT to display messages
SELECT 'Hello World';
SELECT 'MySQL is fun!';
```



★ COMMENTS — TWO STYLES

Anything starting with `--` is a single-line comment and is ignored by MySQL. `/*` like this `*/` wraps a multi-line comment. Use comments generously: they remind future-you what past-you was thinking.



02

Creating & Using Databases

Before you can create tables, you need a database to put them in. Three commands cover the entire lifecycle: create it, switch to it, and (carefully) drop it.

★ THREE RULES TO INTERNALISE FIRST

- **SQL keywords are written in UPPERCASE** — by convention, not by requirement. It just makes code far easier to read.
- **Every SQL statement ends with a semicolon (;)** — this tells MySQL where one statement stops and the next begins.
- **A database holds tables, queries, views, and other objects** — it's the container for everything else.

BASIC SYNTAX

```
CREATE DATABASE name_of_database;  
USE name_of_database;  
DROP DATABASE [IF EXISTS] name_of_database;
```

PRACTICAL EXAMPLES

```
CREATE DATABASE companyHR;  
USE companyHR;  
DROP DATABASE IF EXISTS wrongDB;
```

★ WHY 'USE' MATTERS

The USE statement tells MySQL which database to work with. Without it, MySQL doesn't know which database your subsequent CREATE TABLE or SELECT commands belong to — and you'll get errors. Always USE before you build.

★ SAFETY TIP — 'IF EXISTS'

Adding IF EXISTS to a DROP DATABASE prevents an error if the database isn't there.



Without it, MySQL throws an error and your script halts. With it, the command just succeeds silently and your script keeps running. A small safety net that pays off in automation.



03 Tables, Rows & Columns

A database is a collection of tables. A table is where the actual data lives. Get comfortable with these three words — they are the building blocks of everything you'll do in SQL.

★ THE TABLE VOCABULARY

- **Table** — a structured grid of data, like a spreadsheet. Has a name (e.g. employees).
- **Row** — also called a record. One entry in the table — for example, one employee.
- **Column** — one field across all rows. All values in a column share the same data type.
- **Data type** — defines what kind of value a column can hold (text, integer, date, etc.).
- **Constraint** — optional rules a column must follow (e.g. "cannot be empty").

CREATE TABLE SYNTAX

```
CREATE TABLE table_name (  
    column_name1 datatype [constraints],  
    column_name2 datatype [constraints],  
    ...  
    [table constraints]  
);
```

EXAMPLE — EMPLOYEES TABLE

id	em_name	gender	age
1	Amit Sharma	Male	28
2	Priya Verma	Female	24
3	Rahul Singh	Male	30

★ GOOD TABLE DESIGN MATTERS



Spend a few extra minutes thinking about table design before you create one. Use clear column names. Pick the right data type. Add constraints where they belong. A well-designed table is easier to query, easier to update, and far less likely to surprise you later.



04 Important Data Types

Every column needs a data type. Choosing the right one makes your queries faster, your data safer, and your storage smaller. Here are the ones you'll use most often.

TEXTUAL DATA TYPES

CHAR(size)

Fixed-length string. Good for fixed-size values like gender ('M', 'F') or state codes. Example: CHAR(1).

VARCHAR(size)

Variable-length string. Flexible and good for names, addresses, and any text. Example: VARCHAR(255).

NUMERIC DATA TYPES

INT

Whole numbers (positive, negative, or zero). Use for IDs, ages, counts. Example: INT.

FLOAT

Approximate decimal numbers (single precision). Use for scientific values. Example: FLOAT.

DOUBLE

Approximate decimal numbers (double precision). Larger range. Example: DOUBLE.

DECIMAL(m,d)

Exact decimal with fixed precision. Best for money — never lose a cent. Example: DECIMAL(8,2).

★ MINI EXAMPLES — DATA TYPE IN ACTION

- **Gender** → CHAR(1) → 'M'
- **Full Name** → VARCHAR(255) → 'John Doe'
- **Age** → INT → 25
- **Price** → DECIMAL(8,2) → 199.99



05

Date/Time Types & Constraints

Two more things complete the column toolkit: types for handling time, and constraints that enforce rules on what data is allowed in.

DATE & TIME DATA TYPES

★ THE FIVE TIME TYPES

- **YEAR** — stores just a year value (e.g. 2025).
- **DATE** — stores a date in format YYYY-MM-DD.
- **TIME** — stores a time in format HH:MM:SS.
- **DATETIME** — stores both date and time together (date + time).
- **TIMESTAMP** — often used to store created-time. Useful for tracking when a row was inserted.

COLUMN CONSTRAINTS

★ THE FIVE CONSTRAINTS YOU'LL USE MOST

- **NOT NULL** — value is required; the column cannot be empty.
- **UNIQUE** — every value in the column must be different from every other.
- **DEFAULT** — uses a preset value if none is provided when inserting a row.
- **PRIMARY KEY** — uniquely identifies each row in the table.
- **AUTO_INCREMENT** — value increases automatically with each new row inserted.

EXAMPLE — ALL TOGETHER

```
date_created  TIMESTAMP  NOT NULL  DEFAULT NOW();
```



06 Keys & Relationships

This is where SQL becomes powerful. By linking tables to each other using keys, you can model entire businesses — and answer questions no single table could on its own.

DEFINITION

Primary Key

A column whose value **uniquely identifies each row** in a table. Usually called id. No two rows can share the same primary key.

DEFINITION

Composite Primary Key

When a single column isn't enough, a composite primary key uses **two or more columns together** to uniquely identify each row.

DEFINITION

Foreign Key

A column that **links one table to the primary key of another table**. This is how relationships between tables are formed.

★ PARENT TABLE & CHILD TABLE

The table containing the primary key is the parent table. The table containing the foreign key is the child table. For example: employees (parent) holds the canonical list of people, while mentorships (child) references those employees via foreign keys.

REFERENTIAL ACTIONS — WHAT HAPPENS WHEN THE PARENT CHANGES?



★ THREE COMMON CHOICES

- **ON DELETE CASCADE** — deleting a parent row automatically deletes all related child rows.
- **ON UPDATE RESTRICT** — refuses to update the parent key while child rows still depend on it.
- **ON UPDATE CASCADE** — updates the parent key and the matching child values follow.



FOREIGN KEY CONSTRAINTS BLOCK BAD INSERTS

If you try to insert a row into a child table referencing a parent that doesn't exist, MySQL will reject it. **This is a feature, not a bug — it stops your data integrity from breaking silently.**



07 Changing Tables with ALTER TABLE

Real schemas evolve. You add a column, drop one you no longer need, change a datatype. ALTER TABLE is the tool for all of it — and DESCRIBE is how you check your work.

01 RENAME TABLE

Changes the name of an existing table — sometimes you discover a better name after the fact.

```
RENAME TABLE old_name TO new_name;

-- Example:
RENAME TABLE co_employees TO employees;
```

02 ALTER TABLE — FOUR THINGS IT CAN DO

★ THE ALTER TOOLKIT

- **ADD COLUMN** — adds a new column to the table.
- **DROP COLUMN** — removes an existing column.
- **MODIFY COLUMN** — changes the datatype or constraints of a column.
- **ADD/DROP CONSTRAINTS** — manages rules like NOT NULL, UNIQUE, foreign keys.

03 EXAMPLE — MULTIPLE CHANGES AT ONCE

```
ALTER TABLE employees
  DROP COLUMN age,
  ADD COLUMN salary FLOAT NOT NULL AFTER contact_number,
  ADD COLUMN years_in_company INT NOT NULL AFTER salary;
```

04 DESCRIBE — INSPECT TABLE STRUCTURE



Always check the table's current structure before and after altering it. DESCRIBE shows you every column, its datatype, and its constraints.

```
DESCRIBE employees;
```

★ TWO PIECES OF ALTER-TABLE WISDOM

Use AFTER column_name to control where the new column lands — without it, MySQL appends to the end of the table. And always back up important data before altering — a typo in ALTER TABLE can be expensive to undo on a live production table.



08 Insert, Update & Delete Data

Three commands you'll run more than any others: INSERT puts new rows into a table, UPDATE modifies existing rows, and DELETE removes them. They are simple — and dangerous if you get them wrong.

01 INSERT INTO — ADD NEW ROWS

Used to insert new records into a table. You can insert values into all columns or just selected ones. NULL is allowed when a value is missing.

```
INSERT INTO employees
    (em_name, gender, contact_number, salary, years_in_company)
VALUES ('James Lee', 'M', '516-514-6568', 3500, 11);
```

02 UPDATE — MODIFY EXISTING ROWS

Used to modify existing records. The WHERE clause is critical — it limits the update to specific rows. **Always include WHERE.**

```
UPDATE employees
SET contact_number = '516-514-1729'
WHERE id = 1;
```

03 DELETE — REMOVE ROWS

Used to remove records from a table. As with UPDATE, the WHERE clause is critical — it tells MySQL exactly which rows to delete.

```
DELETE FROM employees
WHERE id = 5;
```



WARNING — WHERE IS NOT OPTIONAL

Without a WHERE clause, **UPDATE will modify every row in the table, and DELETE**



will erase every row in the table. Always run a SELECT with your WHERE clause first to confirm what you're about to change — then change one word to UPDATE or DELETE.

★ **FOREIGN KEYS PROTECT YOU**

Foreign key constraints can block invalid inserts and deletes that would break relationships between tables. If MySQL refuses your INSERT or DELETE, that's usually a sign your data integrity rules are working — not that something is broken.



09

Selecting Data Basics

This is the big one. SELECT is the single most-used SQL command in the world. Master these twelve patterns and you'll be able to answer almost any read-only question against a database.

01. Select all columns and rows

```
SELECT * FROM employees;
```

The * (star) means 'every column'. Returns the entire table.

02. Select specific columns

```
SELECT em_name, gender FROM employees;
```

Best practice: name exactly the columns you need — never use * in production code.

03. Aliases (using AS) — rename a column

```
SELECT em_name AS 'Employee Name' FROM employees;
```

AS gives a column a new label in the result. The underlying column doesn't change.

04. LIMIT — restrict number of rows

```
SELECT * FROM employees LIMIT 10;
```

Returns only the first 10 rows. Essential for peeking at large tables.

05. DISTINCT — remove duplicates

```
SELECT DISTINCT department FROM employees;
```

Removes duplicate values from the result. Useful for finding the unique values in a column.

06. WHERE — filter with operators

```
SELECT * FROM employees WHERE salary >= 50000;
```

Operators you can use: = != > < >= <=

07. BETWEEN — select within a range (inclusive)

```
SELECT * FROM employees WHERE salary BETWEEN 40000 AND 80000;
```



Equivalent to `salary >= 40000 AND salary <= 80000` — but easier to read.

08. LIKE — pattern matching

```
SELECT * FROM employees WHERE em_name LIKE 'A%';
```

% matches any number of characters. _ matches any single character. Example above: names starting with 'A'.

09. IN and NOT IN — membership tests

```
SELECT * FROM employees WHERE department IN ('HR','IT','Sales');
```

```
SELECT * FROM employees WHERE department NOT IN ('HR','IT');
```

Cleaner than chaining OR conditions when you have a fixed list.

10. AND / OR — combine conditions

```
SELECT * FROM employees WHERE department = 'IT' AND salary > 60000 OR  
experience >= 5;
```

Use parentheses () to control evaluation order when mixing AND with OR.

11. ORDER BY — sort results

```
SELECT * FROM employees ORDER BY salary DESC;
```

DESC = descending order. Omit DESC (or use ASC) for ascending.

12. Subquery — a query inside another query

```
SELECT em_name FROM employees WHERE salary > (SELECT AVG(salary) FROM  
employees);
```

The inner query runs first, then the outer query uses its result. Above: everyone earning more than the company average.

★ WHAT'S NEXT?

You now have the foundation. From here, the journey continues into aggregate functions (SUM, AVG, COUNT), GROUP BY and HAVING, joining tables, views, and triggers. Each one builds on the SELECT skills you just learned — practice these



patterns first, and the rest will follow naturally.



Better queries. Better data.

SQL isn't just a language — it's the literacy of the modern data world. Practice these ten chapters on a real database — write queries every day for a week — and you'll feel the language click into place.

★ BRIGHTLEARN • SQL MASTER GUIDE ★